

BoundedLPRQInspired: riduzione bounded exact-turn del PRQ in Mojo

Documentazione tecnica per attività di tesi

Progetto MoStream - 22 luglio 2026

Questa relazione documenta BoundedLPRQInspired, un prototipo MPMC bounded e riusabile ispirato al protocollo di cella della Portable Ring Queue usata dalla LPRQ di Raed Romanov e Nikita Koval. La LPRQ pubblicata collega dinamicamente segmenti PRQ ed è unbounded rispetto al numero di segmenti allocabili. Il prototipo MoStream usa invece un solo ring preallocato, gate CAS e turno FIFO esatto; non implementa skip, UNSAFE e chiusura dei segmenti e non eredita la proprietà lock-free della LPRQ. I test eseguiti non hanno rilevato perdita, duplicazione o errori di riuso, ma il throughput è inferiore a tutte le alternative migliori: zero vittorie su 18 configurazioni e distacco geometrico medio di 3,63 volte dal vincitore. Il risultato riguarda questa riduzione bounded, non le prestazioni della LPRQ originale.

Obiettivo dello studio

MoStream usa code bounded per comunicare fra stage e per strutture del runtime. L'obiettivo dell'esperimento è verificare se alcune idee della LPRQ possano essere trasferite a una coda con memoria fisica e occupazione logica limitate, con payload Mojo generico Optional[T].

Il risultato implementato si chiama deliberatamente BoundedLPRQInspired. Non è corretto chiamarlo LPRQ bounded senza qualificazione, perché la trasformazione rimuove meccanismi essenziali dell'algoritmo pubblicato e modifica le garanzie di progresso.

LPRQ originale e segmenti PRQ

La LPRQ di Raed Romanov e Nikita Koval è una lista dinamica di segmenti Portable Ring Queue. Quando un producer supera il ring, il segmento viene chiuso definitivamente, ne viene collegato uno nuovo e il precedente viene drenato e poi abbandonato. Il numero di segmenti allocabili non è fissato alla costruzione; in questo senso l'algoritmo completo è unbounded.

Il protocollo PRQ può marcare una cella UNSAFE e tollerare ticket saltati perché il segmento non viene riciclato indefinitamente. Il bit può restare permanente: la chiusura sposta il lavoro su un segmento nuovo.

Riutilizzare direttamente lo stesso segmento come ring bounded è pericoloso. Un'operazione sospesa di una generazione precedente potrebbe riprendere dopo un reset e pubblicare nella nuova generazione, producendo un problema ABA. Se UNSAFE restasse permanente, il ring riusato perderebbe progressivamente celle fino al livelock.

Una trasposizione più fedele richiederebbe almeno un pool fisso di segmenti, generation tag per segmento e un protocollo sicuro di reclamation. Non è l'oggetto del prototipo corrente.

Scelta exact-turn

BoundedLPRQInspired elimina skip, UNSAFE e close. Usa ticket monotoni ma ammette un'operazione soltanto sulla cella e sull'epoca esatte associate al ticket FIFO corrente. Il riuso diventa esplicito e bounded, al costo di head-of-line blocking.

La coda è bounded in due sensi:

1. memoria fisica fissa: un solo array di capacity celle, allocato nel costruttore;
2. occupazione logica limitata: il gate impedisce più di capacity ticket producer non ancora consumati.

La capacità deve essere una potenza di due e almeno pari a due. L'indice fisico è ticket & (capacity-1), mentre il ticket completo identifica la generazione.

Struttura dati

I campi principali sono:

cells	array allineato di BoundedLPRQCell[T]
capacity	numero fisso di celle
mask	capacity - 1
head	prossimo ticket consumatore
tail	prossimo ticket producer

I nomi seguono il codice corrente: tail avanza al push e head al pop. Entrambi sono atomici UInt64 racchiusi in strutture padded da 64 byte, così producer e consumer non aggiornano la stessa cache line.

Ogni cella contiene:

state	token del ticket e fase, in un UInt64 atomico
index	epoca/ticket logico della cella, in un UInt64 atomico
data	payload generico Optional[T], non atomico
padding	isolamento al multiplo successivo di 64 byte

L'array è allocato con alignment=64. Il padding riduce il false sharing fra celle adiacenti, ma aumenta il footprint e può esercitare pressione su cache e TLB con capacità elevate. Il costruttore contiene assert compile-time che lo stride di BoundedLPRQCell[T] sia multiplo di 64 byte e che il wrapper dei contatori occupi esattamente 64 byte; un tipo T con layout incompatibile viene quindi rifiutato in compilazione invece di invalidare silenziosamente il padding.

Indice ed epoca della cella

All'inizializzazione, la cella fisica i contiene $\text{index}=i$ e stato EMPTY. Per il ticket producer t , la cella attesa è $\text{cells}[t \ \& \ \text{mask}]$ e deve avere $\text{index}=t$.

Dopo la riserva, il producer cambia index da t a $t+\text{capacity}$. Il consumer del ticket t richiede proprio $\text{index}=t+\text{capacity}$. Dopo il consumo, index non viene azzerato: $t+\text{capacity}$ è già il ticket producer della generazione successiva sulla stessa cella.

vuota per producer t :	$\text{index} = t$
piena per consumer t :	$\text{index} = t + \text{capacity}$
vuota per producer $t+C$:	$\text{index} = t + \text{capacity}$

Lo stato distingue pieno e vuoto quando il valore di index coincide al confine fra due generazioni.

Token e fasi

Tre bit bassi di state rappresentano la fase. I bit superiori contengono un

token derivato dal ticket:

```
ticket_state(t, phase) = ((t + 1) << 3) | phase
phase(state)           = state & 7
```

Le fasi sono:

```
tab( ), box;
Fase Valore Significato
EMPTY 0 nessun proprietario, cella riusabile
RESERVED 1 producer ha rivendicato la cella
WRITING 2 accesso esclusivo al payload in scrittura
FULL 3 payload pubblicato
READING 4 accesso esclusivo al payload in lettura
```

Il token rende lo stato specifico del ticket e riduce l'ABA fra generazioni.

WRITING e READING sono necessari perché Optional[T] non è nello stesso word atomico del token: solo il proprietario della fase può accedere al payload non atomico.

La codifica usa 61 bit effettivi per il ticket. Dopo 2^{61} ticket il token di stato si ripete per wrap-around. Il prototipo considera il limite impraticabile nell'esperimento, ma il wrap non è formalmente supportato o verificato.

Gate bounded dei ticket

Il producer carica $tail=t$ e $head=h$. Rifiuta il tentativo se $h>t$ oppure se $t-h \geq capacity$. Il secondo controllo impedisce di riservare oltre la capacità logica. Se la cella ha $index=t$ e stato EMPTY, un CAS $tail:t \rightarrow t+1$ assegna il ticket.

Il consumer carica $head=h$ e $tail=t$. Se $h \geq t$ non esiste un ticket producer riservato. Anche quando $h < t$, il consumer procede soltanto se la cella di h espone $index=h+capacity$ e l'esatto stato FULL del ticket h . Un CAS $head:h \rightarrow h+1$ assegna il consumo.

Questo gate è una differenza sostanziale dalla LPRQ: PRQ può tollerare buchi e chiudere il segmento; il prototipo bounded deve impedire sovrapprenotazione e attendere il ticket FIFO esatto.

Protocollo push

`try_push` esegue un singolo tentativo prudente:

1. legge $tail$ e $head$ e verifica il limite capacity;
2. verifica $index$ e EMPTY sull'epoca esatta;
3. riserva $tail$ con CAS;
4. invoca `publish_claimed` sul ticket ormai irrevocabile.

`publish_claimed` transita EMPTY $\rightarrow$ RESERVED, cambia $index$ da t a $t+capacity$, transita RESERVED $\rightarrow$ WRITING, scrive data e pubblica lo stato FULL con store RELEASE. Il metodo è statico e riceve soltanto puntatore, capacity, mask, ticket e payload.

```
EMPTY -> RESERVED -> WRITING -> FULL
index t                -> t + capacity
payload                scritto prima della RELEASE FULL
```

`push` è un ciclo sopra `try_push`. Se il ring è pieno o un tentativo perde il CAS, conserva il payload e riprova; dopo 1024 spin chiama `sleep(0.0)`.

Protocollo pop

`try_pop` verifica il ticket $head$ esatto, $index=t+capacity$ e lo stato FULL

contenente il token di t. Solo dopo questi controlli avanza head con CAS, cambia FULL->READING, estrae il payload e pubblica EMPTY con store RELEASE.

FULL -> READING -> EMPTY

payload estratto

index resta t + capacity per il producer della generazione seguente

pop ripete try_pop e applica lo stesso spin-then-yield. Se il producer del ticket head è ritardato dopo la prenotazione di tail, i consumer non possono saltarlo anche se altri producer hanno completato ticket successivi.

Semantica weak delle operazioni try

try_push e try_pop sono single-attempt e possono fallire temporaneamente. Un try_push può restituire il payload anche se la capacità diventerebbe disponibile subito dopo o se perde il CAS contro un altro producer.

try_pop può restituire None perché la coda è realmente vuota, perché il ticket producer precedente è stato riservato ma non ancora pubblicato, oppure perché il CAS su head è stato perso. None non prova quindi in modo linearizzabile che la coda fosse vuota in un preciso istante. L'API va considerata weak try e il chiamante deve essere disposto a ritentare, come fanno i test e il percorso cooperativo di MoStream.

Questa try_pop non è quindi un rimpiazzo diretto per un componente che interpreta subito None come end-of-stream. Tale integrazione richiede un segnale di chiusura separato e polling ripetuto, oppure un risultato tri-state Item, Empty e Pending/Contended.

Una volta vinto il CAS di prenotazione, invece, il ticket non può essere abbandonato: publish_claimed deve completare.

Memory ordering

Il prototipo privilegia prudenza e usa Ordering.SEQUENTIAL per i contatori, l'indice e la maggior parte dei CAS. Le osservazioni di state usano ACQUIRE; la pubblicazione FULL e il ritorno a EMPTY usano RELEASE.

La store RELEASE di FULL rende il payload visibile al consumer che osserva lo stato con ACQUIRE. La fase READING impedisce accessi concorrenti durante l'estrazione. Un lavoro futuro può tentare ordinamenti più deboli, ma soltanto dopo una revisione formale del protocollo e test su architetture weak-memory.

Correzione specifica del lowering Mojo

Una prima versione usava publish_claimed(mut self,...). Con Mojo 1.0.0b1 il lowering applicava copy-in/copy-out dell'intera struct e poteva riscrivere head e tail con snapshot obsoleti. Il sintomo osservato era la prenotazione apparente dello stesso ticket da parte di due thread.

La correzione rende publish_claimed @staticmethod e passa solo i dati necessari. L'IR LLVM del percorso concorrente è stato ispezionato per escludere load/store aggregati della coda. Questa è una correzione del port Mojo, non una proprietà dell'algoritmo LPRQ pubblicato.

API e ciclo di vita

Esempio:

```
from MoStream.Bounded_LPRQ import BoundedLPRQInspired
```

```
var queue = BoundedLPRQInspired[Int](1024)
queue.push(42)
var value = queue.pop()
```

Il payload deve essere Copyable. Sono disponibili push, pop, try_push, try_pop, estimated_len e metodi debug per ticket e celle piene. estimated_len carica head e tail, calcola tail-head e limita il risultato all'intervallo fra zero e capacity; resta uno snapshot approssimativo sotto concorrenza.

Move e distruzione sono supportati soltanto in quiescenza, senza operazioni concorrenti attive e senza thread che conservino ticket o puntatori alle celle. Spostare o distruggere la coda mentre è in uso causerebbe invalidazione dei puntatori e non rientra nel contratto verificato.

Garanzie di progresso

BoundedLPRQInspired non è lock-free e non eredita la prova di progresso della LPRQ. Un producer può vincere il CAS su tail e venire sospeso prima di FULL. Il consumer head non può saltare quel ticket; il ring può quindi subire head-of-line blocking e, una volta esauriti gli slot successivi, il progresso globale dipende dal thread sospeso.

Lo yield periodico riduce lo spin aggressivo ma non risolve la dipendenza. La FIFO esatta è precisamente il compromesso usato per rendere sicuro il riuso di un singolo ring senza il protocollo di chiusura PRQ.

Verifica di correttezza

Il test sequenziale verifica riempimento, rifiuto al nono elemento con capacità otto, FIFO, coda vuota, 10000 generazioni e riempimento dopo wrap-around.

Due test 4P-4C inviano 100000 messaggi per producer con capacità 2, 4, 8 e 16: uno usa push/pop bloccanti, l'altro ritenta le weak try_push/try_pop. Entrambi verificano count, checksum $N(N-1)/2$, coda vuota dopo i sentinel e una sola occorrenza di ciascun ID mediante 400000 contatori atomici per-ID. Quest'ultimo oracolo rileva anche combinazioni di duplicati e perdite che conservano somma e conteggio complessivi.

La campagna documentata comprende inoltre 20 esecuzioni aggiuntive dei binari e il controllo dell'IR LLVM. Il 22 luglio 2026 i tre test principali sono stati rieseguiti e hanno prodotto:

```
PASS: sequential boundedness and FIFO correctness
PASS: blocking API capacity 2, 4, 8, 16
PASS: concurrent exact-once correctness
PASS: try API capacity 2, 4, 8, 16
PASS: concurrent try exact-once correctness
```

I test sono evidenza sperimentale, non una prova formale di linearizzabilità o di correttezza su tutti gli interleaving e memory model.

Metodo sperimentale

Il benchmark è stato eseguito il 22 luglio 2026 su Intel Xeon Gold 5512U, 28 core e 56 thread hardware, singolo nodo NUMA, Linux x86-64 e Mojo 1.0.0b1.

Configurazione:

- 100000 messaggi per producer e 10 ripetizioni;
- topologie 1P-1C, 2P-2C, 4P-4C, 8P-8C, 1P-8C e 8P-1C;
- capacità 16, 1024 e 65536;

- ordine delle implementazioni ruotato fra le ripetizioni;
- count e checksum verificati in ogni esecuzione;
- media, deviazione standard campionaria e intervallo di confidenza 95 percento.

Le baseline sono CAS-MPMC, FAA, Rigtorp, Padded-FAA e Hybrid con soglie 1, 2, 4 e 8. La macchina non è stata isolata e i thread non sono stati pinnati. I risultati sono quindi comparativi locali e richiedono replica controllata prima di una pubblicazione definitiva.

Risultati bilanciati a capacità 1024

La tabella riporta throughput medio in milioni di messaggi al secondo dal CSV rigenerato results_bounded_lprq.csv. Best Hybrid è il massimo delle quattro soglie; l'ultima colonna confronta il vincitore con BLPRQ.

```
tab( ),box;
Thread CAS FAA Rigtorp Padded Best H BLPRQ Winner/BLPRQ
1P-1C 5.023 6.690 4.410 4.993 6.367 2.219 FAA 3.01x
2P-2C 4.528 7.237 7.880 9.085 5.746 3.315 Padded 2.74x
4P-4C 3.841 8.692 8.645 10.842 7.132 3.012 Padded 3.60x
8P-8C 2.530 9.416 8.659 10.588 6.678 2.159 Padded 4.90x
```

Gli intervalli 95 percento di BLPRQ sono rispettivamente +/-0.031, +/-0.009, +/-0.066 e +/-0.034 Mmsg/s. Gli intervalli completi di ogni baseline sono conservati nel CSV.

Risultati sull'intera matrice

Vittorie per massimo throughput medio:

```
tab( ),box;
Implementazione Vittorie su 18
Padded-FAA 14
FAA 2
Rigtorp 2
BLPRQ 0
CAS 0
Hybrid-1, 2, 4, 8 0
```

La migliore implementazione di ciascuna configurazione è fra 2.50 e 5.75 volte più veloce di BLPRQ. La mediana del rapporto è 3.57x e la media geometrica 3.63x.

Media geometrica dei rapporti rispetto a BLPRQ:

```
tab( ),box;
Queue Rapporto Queue Rapporto
CAS 1.53x FAA 2.69x
Rigtorp 2.91x Padded-FAA 3.46x
Hybrid-1 2.43x Hybrid-2 2.17x
Hybrid-4 1.80x Hybrid-8 1.65x
```

Il distacco minimo, 2.50x, si verifica in 2P-2C capacità 16; il massimo, 5.75x, in 1P-1C capacità 65536. Quest'ultimo caso presenta elevata variabilità per Padded-FAA e va replicato su macchina isolata.

Interpretazione del risultato negativo

BoundedLPRQInspired non ottiene vittorie. I costi principali sono:

1. il gate CAS è necessario per impedire la sovrapprenotazione che PRQ gestisce tramite skip e chiusura del segmento;
2. ogni trasferimento attraversa ticket, index, token e più transizioni atomiche;
3. Optional[T] separato richiede WRITING e READING, mentre una rappresentazione

compatta token-puntatore può ridurre passaggi;

4. la FIFO exact-turn elimina gli skip e introduce head-of-line blocking;
5. SEQ_CST, CAS retry e cache coherence crescono con la contesa;
6. padding e due word atomici per cella aumentano memoria e traffico.

Il peggioramento relativo cresce nelle topologie 8P-8C e SPMC, coerentemente con un percorso ricco di CAS e metadati. BLPRQ resta un prototipo utile per studiare perché la chiusura dei segmenti sia strutturale nella LPRQ.

Cosa non dimostrano i numeri

Questi dati non dimostrano che la LPRQ di Romanov e Koval sia lenta. Non è stata implementata né benchmarkata la lista dinamica di segmenti con il protocollo PRQ completo. È stata misurata una riduzione bounded conservativa che sacrifica proprio skip, close e lock-freedom.

Le classifiche sono basate sulle medie di una singola campagna. Intervalli sovrapposti, rumore, frequenza CPU, allocazione dei thread e assenza di pinning limitano inferenze causali. Il throughput non descrive latenza p99, fairness, energia o risposta alla sospensione dei thread.

Limiti di correttezza e API

- Le try sono weak: un fallimento temporaneo non certifica pieno o vuoto.
- Move e distruzione richiedono quiescenza completa.
- Il token si ripete dopo 2^{61} ticket; wrap-around non formalmente supportato.
- Non esiste una prova formale di linearizzabilità del port Mojo.
- Non sono stati esplorati sistematicamente tutti gli interleaving.
- Il lowering corretto è stato verificato per Mojo 1.0.0b1, non per versioni future.
- Un thread sospeso dopo la prenotazione può arrestare il ticket FIFO successivo.
- Payload e capacità differenti possono modificare cache, layout e prestazioni.

Applicabilità a MoStream

Nella matrice corrente Padded-FAA resta la scelta generale più forte, FAA vince due casi semplici e Rigtorp due workload SPMC. BoundedLPRQInspired non è una sostituzione prestazionale consigliata per Communicator o scheduler.

Il prototipo è invece utile come risultato di tesi: mostra empiricamente che fissare un singolo ring e attendere il turno esatto annulla i vantaggi del meccanismo PRQ. Per avvicinarsi all'algoritmo originale senza memoria illimitata serve studiare un pool bounded di segmenti con generazioni e reclamation, non aggiungere altre ottimizzazioni locali al solo ring exact-turn.

Lavoro futuro

1. Formalizzare linearizzabilità e progresso con model checking.
2. Creare uno stress harness con storie concorrenti e verifica offline.
3. Sospendere thread dopo tail CAS e durante WRITING o READING.
4. Misurare p50, p95, p99, fairness, CPU ed energia su host pinnato.
5. Provare ordinamenti meno forti solo dopo analisi memory-model.
6. Progettare un pool fisso di segmenti PRQ con generation tag.
7. Definire reclamation sicura per segmenti riutilizzabili.
8. Confrontare payload atomici compatti e Optional[T] generico.
9. Ripetere su ARM e sistemi NUMA per esporre weak ordering e locality.

Conclusione

BoundedLPRQInspired raggiunge memoria e occupazione limitate e supera i test eseguiti grazie a index per epoca, token per ticket, fasi esclusive e gate CAS. Questa prudenza trasforma però il protocollo in una coda exact-turn non lock-free, con più metadati e operazioni atomiche delle baseline.

Il risultato sperimentale è netto: zero vittorie su 18, Padded-FAA prima in 14 casi e un vincitore mediamente 3.63x più veloce. La conclusione scientificamente corretta non è che LPRQ sia inefficiente, ma che la conversione in un unico ring bounded e riusabile rimuove il meccanismo con cui LPRQ tollera i ticket saltati.

Artefatti e fonti

- MoStream/Bounded_LPRQ.mojo, implementazione exact-turn.
- Tests/test_bounded_lprq.mojo, test sequenziale e generazionale.
- Tests/test_bounded_lprq_concurrent.mojo, test bloccante 4P-4C.
- Tests/test_bounded_lprq_try_concurrent.mojo, test weak try 4P-4C.
- Benchmarks/QueueBenchmark/results_bounded_lprq.csv, dati aggregati.
- Benchmarks/QueueBenchmark/analyze_lprq_results.py, riepilogo riproducibile.
- Raed Romanov e Nikita Koval, The State-of-the-Art LCRQ Concurrent Queue Algorithm Does NOT Require CAS2, PPOPP 2023, DOI 10.1145/3572848.3577485.
- Artifact ufficiale: <https://zenodo.org/records/7337237>.